



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira

Disciplina: Teoria dos Grafos



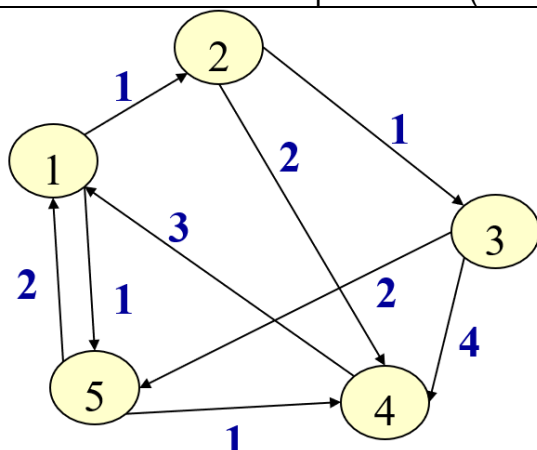
Implementação

Algoritmo de Dijkstra ou Bellmann-Ford ou Floyd (Grupo do Projeto)

Nome do(a) aluno(a)	RA
Edmilson Li Quansang	10425514
João Paulo Berto Massabki	10425593
Pietro Caffettani	10425628

Utilizando a classe grafo que faz uso da matriz de adjacência ou lista de adjacência, disponibilizada em aula, na linguagem de programação que está sendo utilizada para o desenvolvimento do seu projeto, realizar a implementação dos algoritmos de Dijkstra ou Bellmann-Ford ou Floyd.

Realizar pelo menos dois testes, sendo **NECESSARIAMENTE**:

Grafos a serem utilizados para testes (Necessariamente)																													
	<table><tr><th></th><th>1</th><th>2</th><th>3</th><th>4</th></tr><tr><th>1</th><td>0</td><td>20</td><td>30</td><td>50</td></tr><tr><th>2</th><td>20</td><td>0</td><td>40</td><td>15</td></tr><tr><th>3</th><td>30</td><td>40</td><td>0</td><td>15</td></tr><tr><th>4</th><td>50</td><td>15</td><td>15</td><td>0</td></tr></table>					1	2	3	4	1	0	20	30	50	2	20	0	40	15	3	30	40	0	15	4	50	15	15	0
	1	2	3	4																									
1	0	20	30	50																									
2	20	0	40	15																									
3	30	40	0	15																									
4	50	15	15	0																									
Grafo do Material da Aula	Grafo da Atividade Solicitada Anteriormente	Atividade																											

Colocá-los no relatório a seguir. Em uma seção final do relatório, denominada **Apêndice**, inclua o código fonte para os algoritmos desenvolvidos, devidamente documentado.

Os códigos-fontes desenvolvidos para esta atividade devem OBRIGATORIAMENTE ser baseados no pseudocódigo do material da aula. Caso não seja feito desta forma, a atividade será zerada.

Todos os códigos fontes necessários para a execução dos testes e o relatório devem ser **OBRIGATORIAMENTE** encaminhados pelo Ambiente Virtual. **FICAR ATENTO!**



UNIVERSIDADE PRESBITERIANA MACKENZIE
Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira

Disciplina: Teoria dos Grafos

Resolução



Aplicando Dijkstra para Grafo Direcionado do Exemplo:

Distância mínima da origem 3 para:

1-> 3.0

2-> 4.0

4-> 3.0

5-> 2.0

Rota da 3 para 1: 3 -> 5 -> 1

Rota da 3 para 2: 3 -> 1 -> 5 -> 2

Rota da 3 para 4: 3 -> 5 -> 4

Rota da 3 para 5: 3 -> 5

Aplicando Dijkstra para Grafo Não Direcionado do Exemplo:

Distância mínima da origem 3 para:

1-> 3.0

2-> 4.0

4-> 3.0

5-> 2.0

Rota da 3 para 1: 3 -> 5 -> 1

Rota da 3 para 2: 3 -> 1 -> 5 -> 2

Rota da 3 para 4: 3 -> 5 -> 4

Rota da 3 para 5: 3 -> 5

Conforme vemos na imagem para tanto o Grafo Direcionado quanto para o Não Direcionado o Algoritmo foi capaz de indentificar as menores distâncias do ponto origem para os demais corretamente, assim como fazer as rotas da origem para os demais.



Apêndice

Como a base do código também inclui Classes como Builders e Grafos, farei a entrega desses apenas no arquivo compactado através do Ambiente Virtual. Aqui colocarei só os métodos de uma classe chamada “Dijkstra” responsável pela implementação do algoritmo.

```
public void dijkstra_algoritmo(Grafo grafo, int origem){
    if(origem >= grafo.getN()) return;
    origem -= 1;

    Float[] d = new Float[grafo.getN()];
    for (int i = 0; i < grafo.getN(); i++) {
        if (i == origem) {
            d[i] = 0f;
        } else {
            d[i] = Float.POSITIVE_INFINITY;
        }
    }

    List<Integer> aberto = new ArrayList<>();
    List<Integer> sucessores;
    Integer[] rot = new Integer[grafo.getN()];

    for (int i = 0; i < grafo.getN(); i++) {
        rot[i] = 0;
        aberto.add(i);
    }

    while (!aberto.isEmpty()) {
        int r = aberto.getFirst();

        for (int i = 0; i < grafo.getN(); i++) {
            if(!aberto.contains(i)) continue;

            if(aberto.size() == 1){
                r = aberto.getFirst();
            }
            else if (d[i] < d[r]) {
                r = i;
            }
        }

        aberto.remove(aberto.indexOf(r));
        sucessores = this.rotas_de_v(r, grafo);
    }
}
```



UNIVERSIDADE PRESBITERIANA MACKENZIE

Faculdade de Computação e Informática

Prof. Dr. Ivan Carlos Alcântara de Oliveira

Disciplina: Teoria dos Grafos



```
        for (int i = 0; i < sucessores.size(); i++) {
            float p = Math.min(d[sucessores.get(i)], (d[r] +
grafo.getAdj()[r][sucessores.get(i)]));
            if (p < d[sucessores.get(i)]) {
                d[sucessores.get(i)] = p;
                rot[sucessores.get(i)] = r;
            }
        }
    }

    System.out.println("Distância mínima da origem " + (origem+1) + " para: ");
    for(int i=0; i<grafo.getN(); i++){
        if(i == origem) continue;
        System.out.println(i+1 + "-> " + d[i]);
    }

    this.print_rot(rot, grafo, origem);
}
```

```
public List<Integer> rotas_de_v(int v, Grafo grafo) {
    List<Integer> rotas = new ArrayList<>();
    for (int i = 0; i < grafo.getN(); i++) {
        if (grafo.getAdj()[v][i] != Float.POSITIVE_INFINITY) {
            rotas.add(i);
        }
    }
    return rotas;
}
```

```
public void print_rot(Integer[] rot, Grafo grafo, int origem){
    for (int i = 0; i < grafo.getN(); i++) {
        if(origem == i) continue;
        int k = i;
        System.out.print("\n\n Rota da " + (origem+1) + " para " + (i+1) + ": " +
(origem+1));
        while (rot[rot[k]] != 0) {
            System.out.print(" -> " + (rot[k]+1));
            k = rot[k];
        }
        System.out.print(" -> " + (i+1));
    }
}
```